KRIPYA TECHNICAL EVALUATION · INFRASTRUCTURE MANAGEMENT TOOL

# Infra Portal

A self-service portal for requesting servers on Proxmox VE and AWS

Muthu · October 2026 · github.com/Muthujar/infra-portal

THE PROBLEM

# Getting a server today means a ticket and a wait

- A developer raises a ticket and waits days for the DevOps team.
- Servers are built by hand, so no two are quite the same.
- There is no single record of who asked for what, who approved it, and what was built.
- Test servers are forgotten and keep using capacity.

**Goal:** a catalog, an approval step and automation, in the spirit of VMware vRealize Automation (Aria).

THE SOLUTION

# Four steps from request to ready server

1 · DEVELOPER

## Request

Picks a server from the catalog, names it, chooses DEV / QA / PROD and a lease.

2 · API

## Check

Validates input, checks the team quota and saves the request with an audit entry.

3 · TEAM LEAD

## Approve

Approves DEV/QA for the team. PROD needs an admin. Terraform code is generated.

4 · WORKER

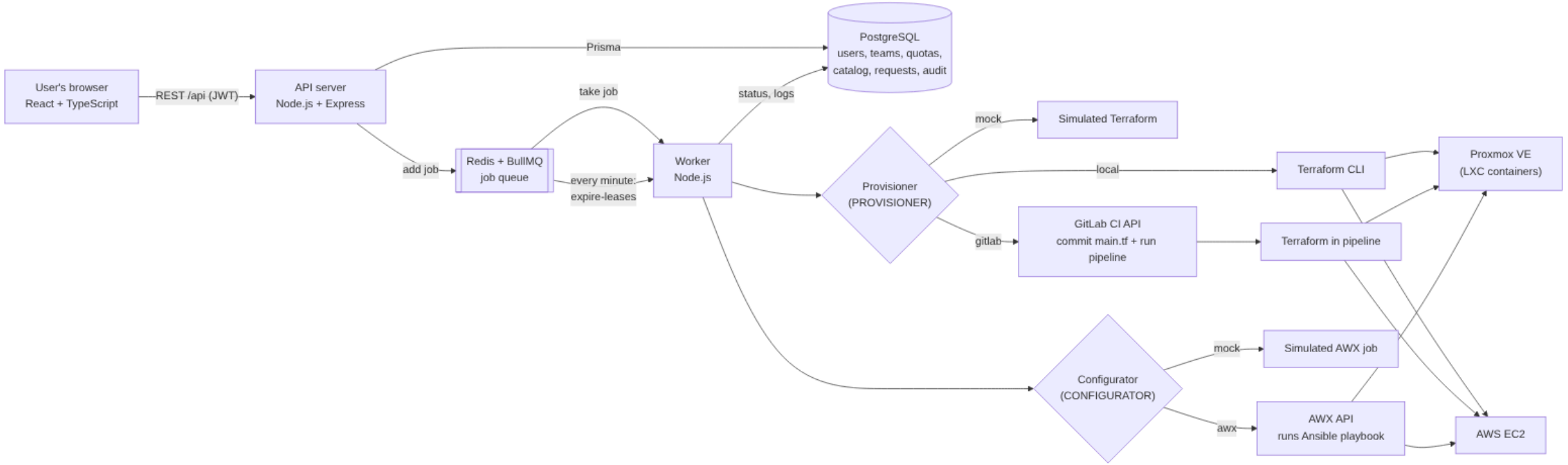
## Build

Runs Terraform, then the AWX (Ansible) job. The UI shows every status live.

Every change is recorded in the audit log. Servers with a lease are deleted automatically when it ends.

ARCHITECTURE

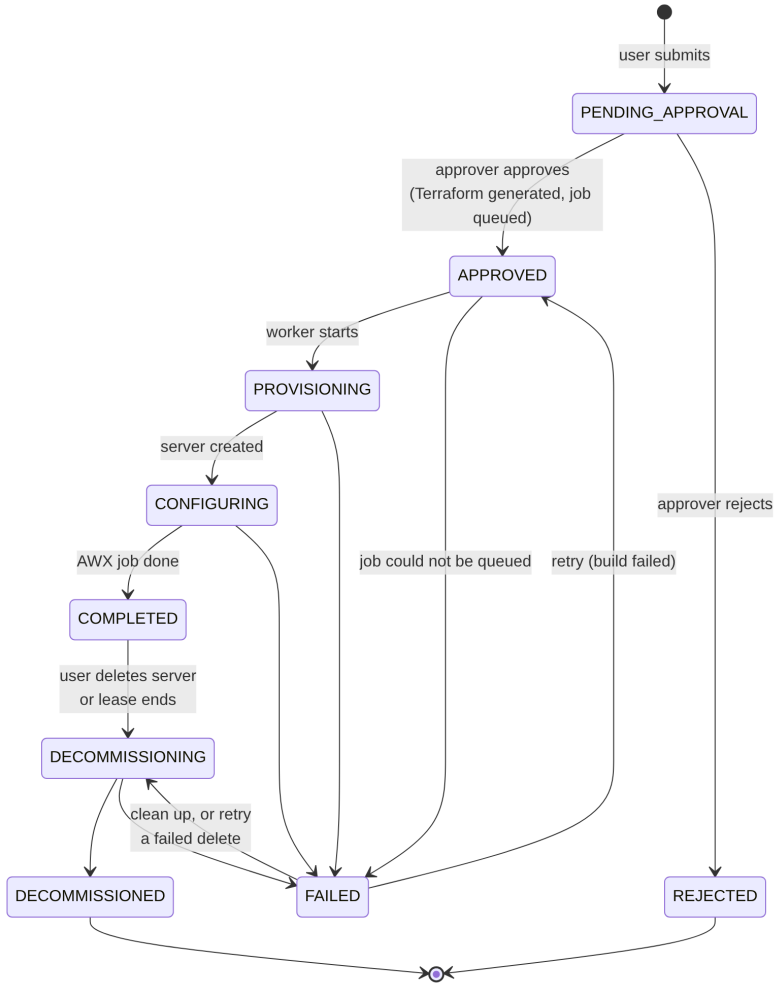
# API, queue and worker keep slow work out of the web request




React UI · Node.js + Express API · PostgreSQL (Prisma) · Redis + BullMQ queue · worker with pluggable provisioner (mock / local Terraform / GitLab CI) and configurator (mock / AWX)

REQUEST LIFE CYCLE

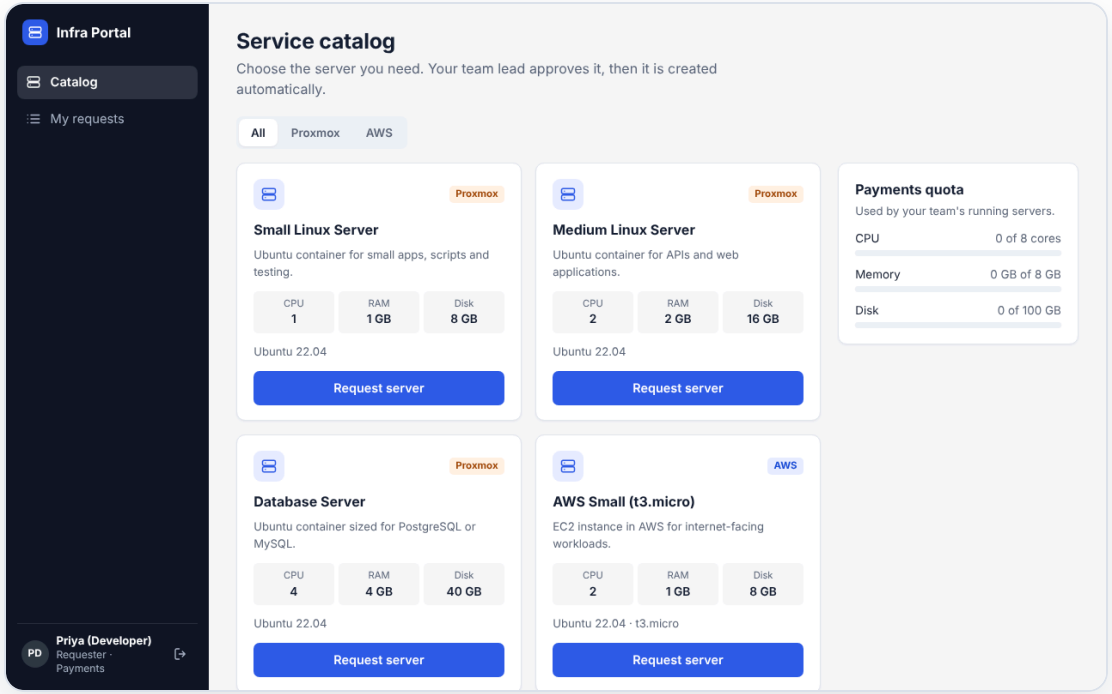
# One state machine controls every status change



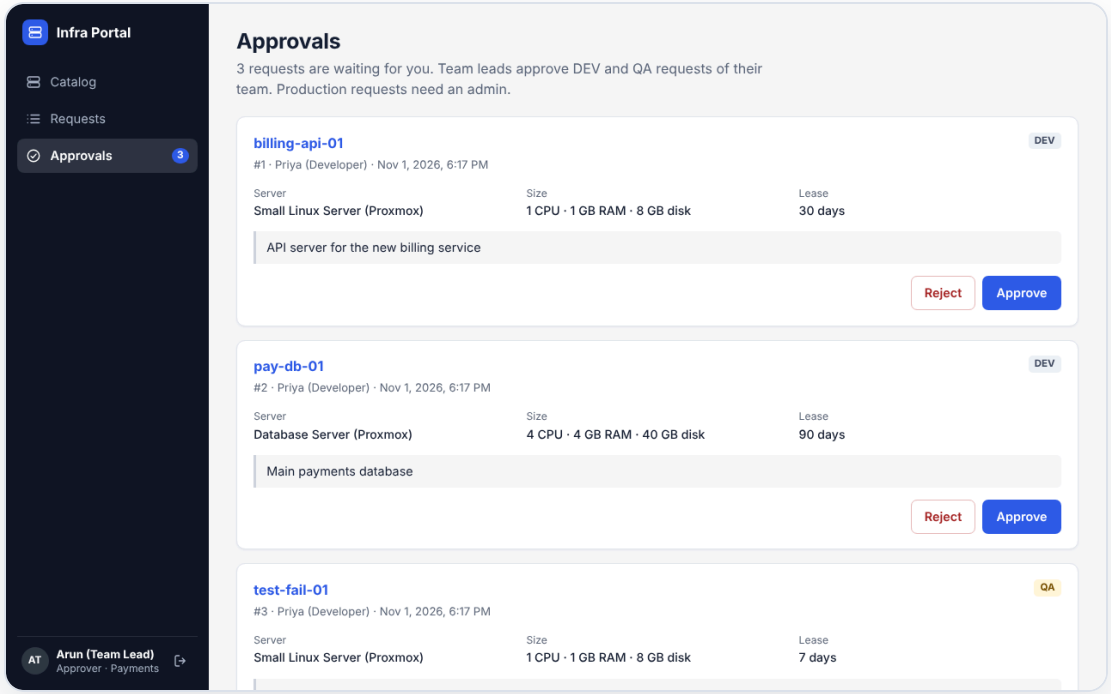

- Only allowed moves are accepted. A rejected request can never be built.
- The update succeeds only if the status has not changed meanwhile, so two approvers or two workers cannot act at once.
- Each change writes who, when, from and to into the audit log.

DEMO · REQUEST AND APPROVE

# Catalog with team quota, and the approval queue



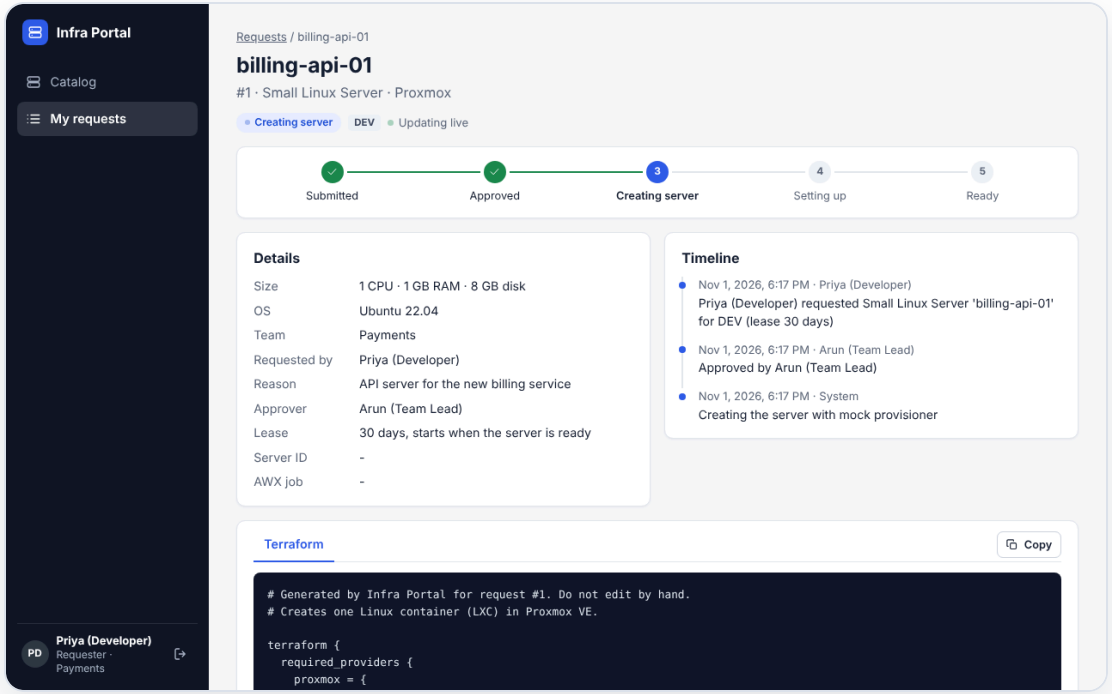

Developer: catalog and remaining team quota



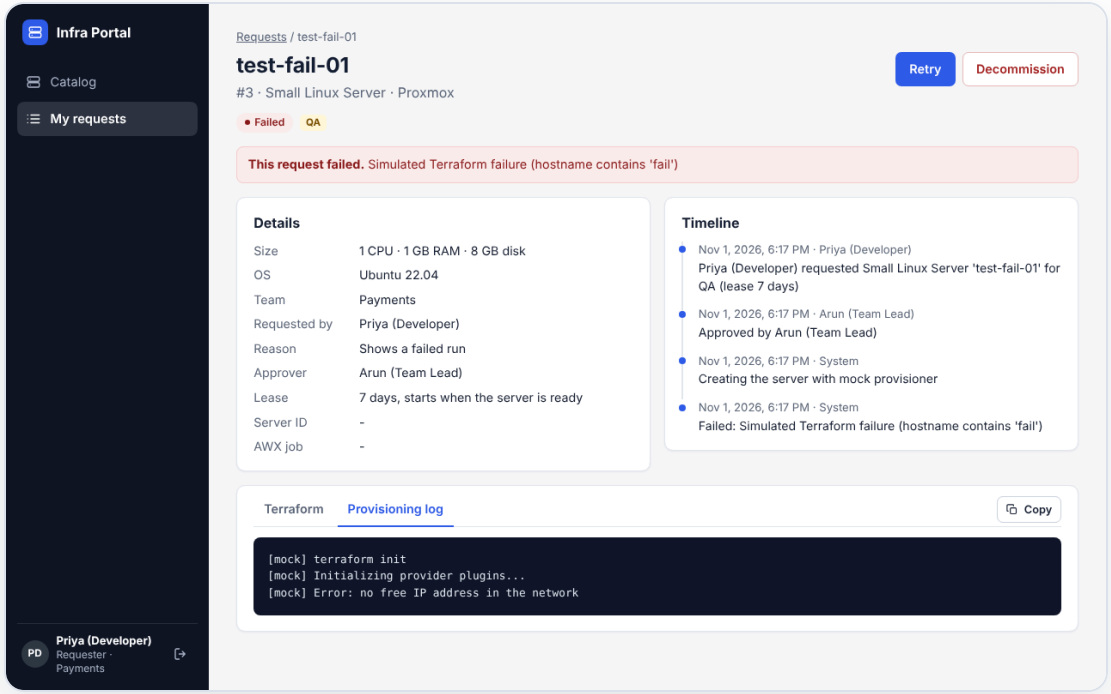

Team lead: approve or reject with a comment

DEMO · PROVISIONING

# Live status, timeline and the generated Terraform




Creating the server: progress bar, timeline, main.tf




Failure path: error, log, Retry and Decommission

GOVERNANCE

# Who can do what, and what it costs

### Roles (RBAC)

Requester, approver, admin. Nobody approves their own request. PROD needs an admin. Requesters see only their own requests.

### Team quotas

CPU, RAM and disk limits per team, checked on request, on approval and on retry. Admins edit them.

### Audit log

Every action with actor, time and status change. Visible on each request and in full to admins.

### Leases

7, 30 or 90 days or no expiry. The worker deletes expired servers every minute; owners can extend.

### Secrets

Proxmox and AWS credentials go to Terraform as environment variables, never into main.tf or the database.

### Validation

Every input is checked with Zod: hostname format, unique names, sizes, comments.

HONEST SCOPE

# What is real and what is simulated

| PART | STATUS | SWITCH |
|---|---|---|
| UI, API, database, RBAC, quotas, approvals, leases, audit, queue, worker | Real | always on |
| Terraform generation (Proxmox LXC, AWS EC2) | Real | validated in CI |
| Creating containers in Proxmox VE | Real | PROVISIONER=local |
| AWS EC2 apply | Validate only | AWS_APPLY_ENABLED=true |
| GitLab CI pipeline | Mock in demo | PROVISIONER=gitlab |
| AWX / Ansible job | Mock in demo | CONFIGURATOR=awx |
| SAML / LDAP login | Not built | next step: Keycloak |

QUALITY

# Tested and checked on every push

- **34 automated tests** (Vitest + Supertest) on a real PostgreSQL database
- Full life cycle, approval rules, validation, quotas, leases, state machine, Terraform generator, GitLab connector
- Connector pattern: the same tests run the worker steps with the mock provisioner

- **GitHub Actions CI**, four jobs:
  - backend: typecheck, tests, build
  - frontend: lint, build
  - terraform: fmt + validate of the generated files
  - docker: compose up, then a smoke test drives a request to Ready

NEXT STEPS FOR PRODUCTION

# From prototype to a real platform

- Single sign-on with SAML / LDAP (Keycloak or Active Directory), directory groups mapped to roles
- Remote Terraform state (GitLab or S3) instead of local folders
- Secrets in a vault (HashiCorp Vault, AWS Secrets Manager)
- Email or Slack notifications on approval, completion and before a lease ends
- Run the real GitLab pipeline and AWX job template against the company's own instances

Setup and demo: README.md · docs/setup-windows-proxmox.md · docs/demo.md · docs/architecture.md